01 / PROJECT DESCRIPTION

SkillSwap

A campus skill-sharing application for one-to-one peer learning.

Dardanian Devs

Redon / Rinor / Granit / Leanda

Prepared for Professor Valon Raca | SWEN383

The problem

Students often know classmates who need help, but there is no clear place to see who can teach a particular skill or arrange a session. Useful offers get lost in group messages, and the final time and status of a meeting can be unclear.

The proposed application

SkillSwap gives students one place to create a profile, describe skills they can teach, search for help, and request a learning session. A student can act as both a learner and a tutor using the same account. The tutor accepts or declines a proposed time, and both participants can track their sessions.

EXAMPLE JOURNEY

Rinor searches for Java help, opens Redon's listing, and requests a 45-minute session. Redon accepts if both students are free. The session appears in their dashboards, and Redon marks it completed after it ends.

Semester-sized scope

The first release serves one university community. It focuses on free, individual sessions and a complete booking workflow. Sharing is voluntary: teaching does not create a credit balance or require a direct exchange.

Report contents	Pages
Scope and requirements; architecture; domain model; booking rules	2-5
Four UML sequence diagrams covering the core workflows	6-9
Design risks and planned patterns; validation and references	10-11

02 / SCOPE AND REQUIREMENTS

What the application will do

The minimum release is defined by six functional requirements.

ID	Requirement	Completion criterion
FR-01	Accounts and profiles	A student can register, sign in/out, and edit their name, bio, and learning interests. Email is unique; passwords are stored as hashes.
FR-02	Manage skill offers	A student can create, edit, and archive their own listings with a title, category, description, and availability note.
FR-03	Find skills	Signed-in students can search active listings by title/description and filter by category. Results link to listing and tutor details.
FR-04	Request a session	A learner can propose a future start/end time, meeting details, and a short message on another student's active listing.
FR-05	Respond to requests	The listing owner can accept or decline a pending request. Acceptance checks conflicts for both participants.
FR-06	Manage sessions	Participants can view their requests and history, cancel under the rules on page 5, and see the tutor mark an ended session completed.

Who uses it

Learner: finds a skill and requests help. **Tutor:** publishes a skill and responds to requests. These are roles in a session, not separate account types. Only the two participants can access a session's private details.

Explicitly outside this release

Payments and time credits; ratings and reviews; real-time chat; built-in video calls; AI matching; group classes; recurring bookings; calendar or email integrations; university single sign-on; a public administrator dashboard; and native mobile apps.

Practical boundaries

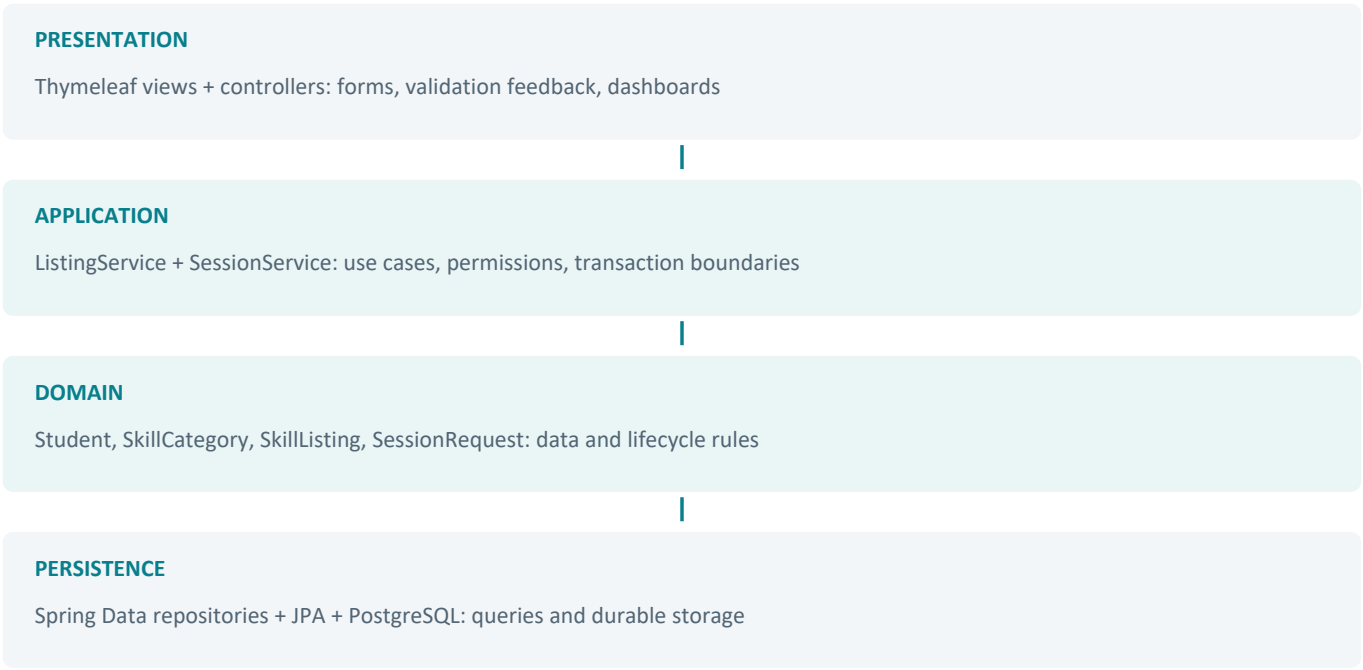
Learning interests are profile text, while searchable listings advertise skills to teach. Categories are seeded by the team. Availability is a descriptive note; the learner proposes a precise time. Meeting details can describe a campus location or an external call link. Rescheduling means cancelling and submitting a new request.

03 / ARCHITECTURE

One application, clear responsibilities

Proposed stack: Java, Spring Boot, Thymeleaf, Spring Data JPA, PostgreSQL, and Maven.

A server-rendered web application keeps the semester project manageable. Spring MVC controllers handle requests and return Thymeleaf views [2]. Spring Security will manage login sessions and CSRF protection. HTML/CSS and small amounts of JavaScript provide the interface; JUnit and database integration tests support validation.



How a request travels

The controller obtains the authenticated student ID and calls a service. The service checks ownership and business rules, loads or changes domain objects, and calls repositories. A service-level transaction encloses each booking change. Only after it commits does the controller redirect to the updated page [3, 4, 5].

Design choices that keep the scope small

One deployable application and one relational database. No message broker, separate API application, or background scheduler is required. Session statuses are represented by an enum and guarded domain methods; a separate class for every status would add little value to this fixed workflow.

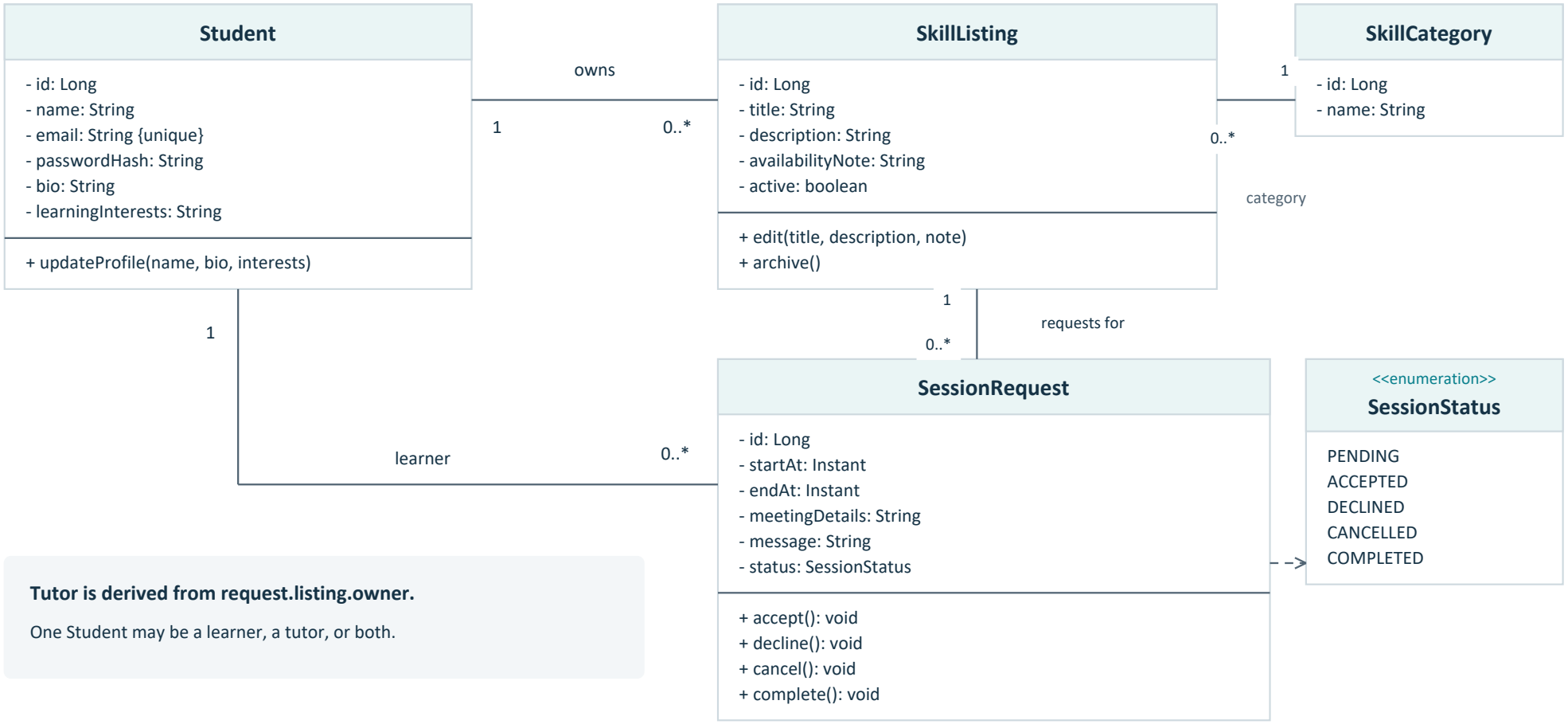
Security boundary

All application pages require login except registration and sign-in. The server derives the acting student from the login session, checks access on every mutation, and never accepts a submitted owner ID as authority. Views escape user text, and contact details are excluded from listing search results.

04 / UML CLASS DIAGRAM

Domain model

CD-01 | Main entities, responsibilities, association roles, and multiplicities.



Reading the model. 1 means exactly one; 0..* means zero or more. Association reference fields are shown by lines; they become foreign keys. The listing owner is the tutor. Plain associations do not imply cascading deletion. Controllers, services, and repositories appear in the architecture and sequences. UML notation follows [1].

05 / BUSINESS RULES

A small, explicit session lifecycle

A request is a proposal. It becomes a booking only when the tutor accepts it.

Current state	Action / authorized student	Next state
PENDING	Accept / tutor, before start; both students free	ACCEPTED
PENDING	Decline / tutor	DECLINED
PENDING	Cancel / learner	CANCELLED
ACCEPTED	Cancel / either participant, before start	CANCELLED
ACCEPTED	Complete / tutor, at or after end	COMPLETED
DECLINED, CANCELLED, COMPLETED	No further transitions	Unchanged

Rules shared by every workflow

BR-01 The authenticated student controls only their own profile/listings. A session is visible only to its learner and tutor.

BR-02 A request must target another student's active listing. Both timestamps are required; start must be in the future and end must be after start.

BR-03 Accepted sessions must not overlap for either participant, whether they appear as learner or tutor. Pending requests do not reserve time.

BR-04 The service enforces the table above. Repeated or stale actions return the current state with an explanation and do not overwrite it.

BR-05 Archiving hides a listing from search and blocks new requests. Existing requests remain available and follow the same lifecycle.

BR-06 Request participants and times are immutable. Timestamps are stored as instants and displayed in the configured campus time zone, shown on every booking form.

OVERLAP RULE

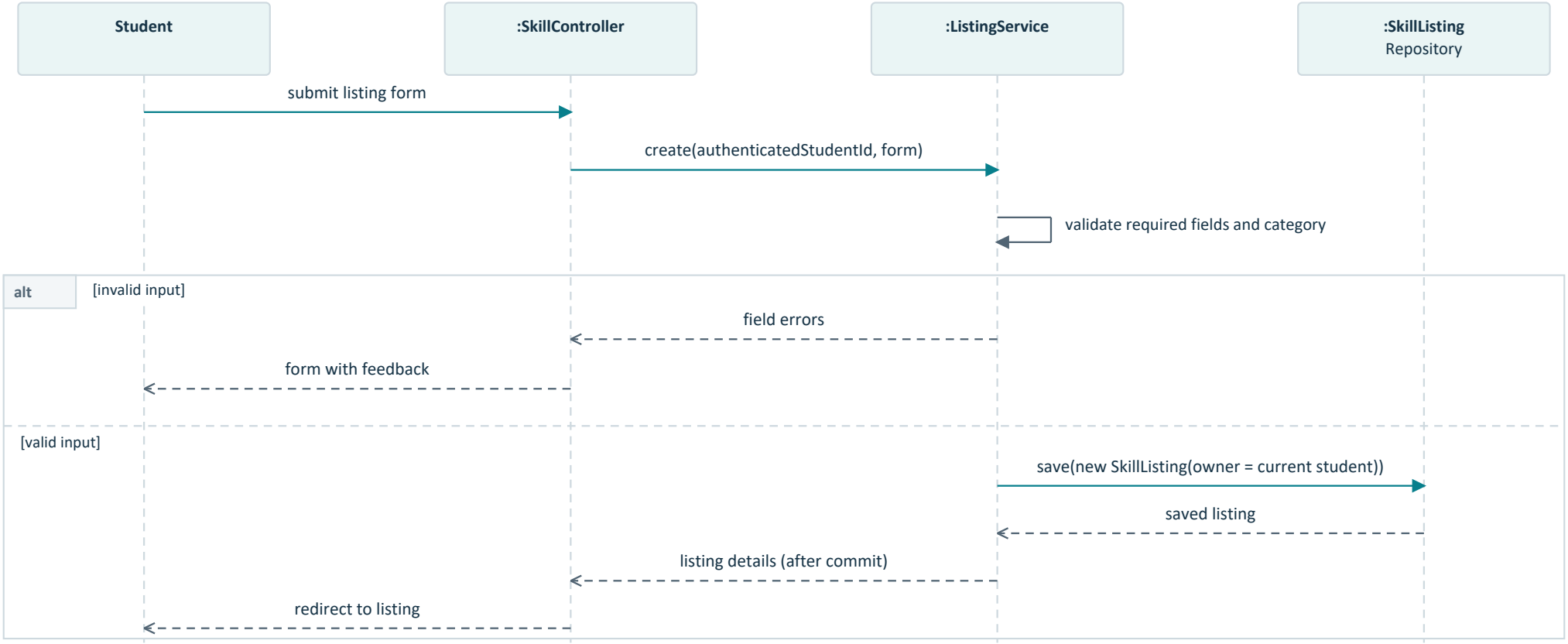
Intervals [start, end) overlap when **existing.start < proposed.end** and **proposed.start < existing.end**. A session ending at 14:00 can be followed by one beginning at 14:00.

No automatic expiry is required: a pending request whose start has passed cannot be accepted, but can still be declined or cancelled. An accepted session stays accepted until the tutor records completion. Completion records a status; it does not verify attendance.

06 / UML SEQUENCE DIAGRAM

Publish a skill offer

SD-01 | FR-02 | Ownership is assigned by the server.



Precondition. The student is signed in. Categories are already available in the form.

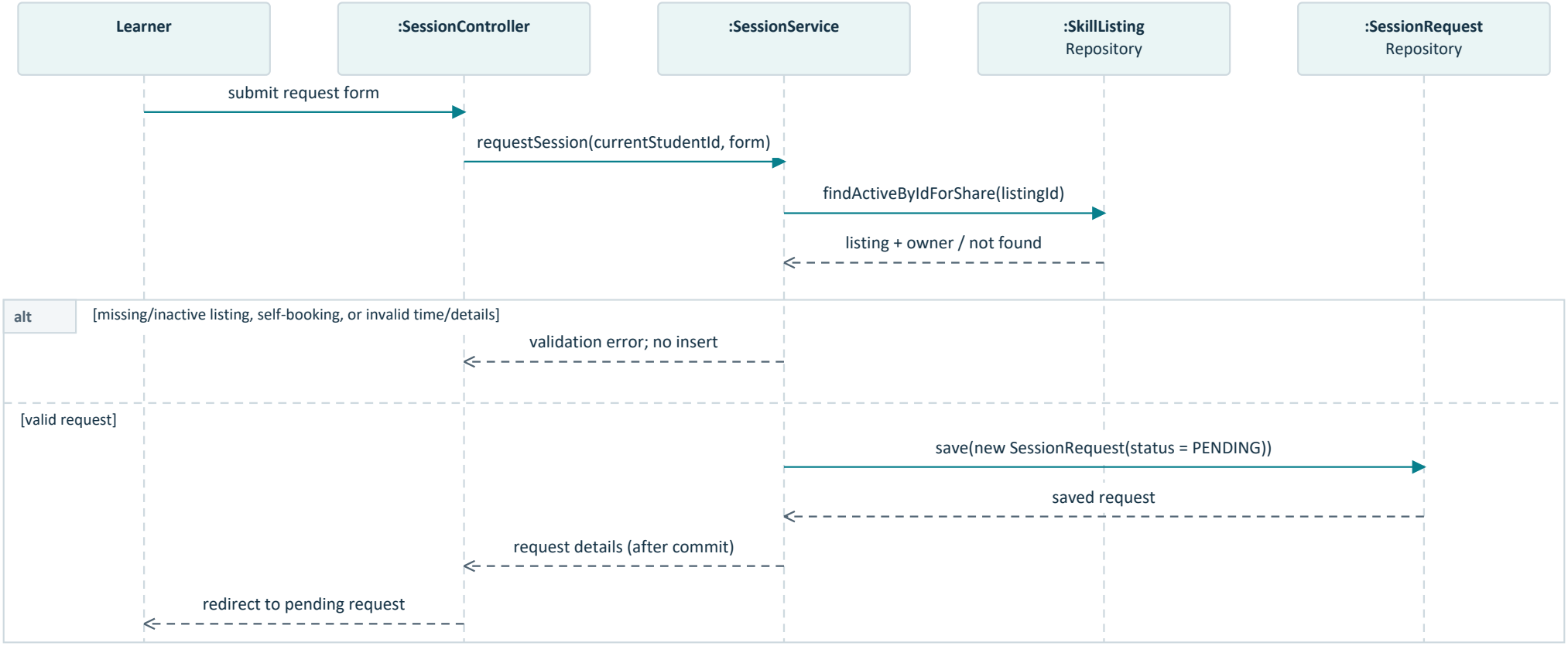
Outcome. On the valid path, an active listing is committed and the controller redirects to its details page.

Failure handling. Invalid fields leave storage unchanged. A persistence failure rolls back the save and displays a retry message.

07 / UML SEQUENCE DIAGRAM

Request a learning session

SD-02 | FR-03, FR-04 | A pending request does not reserve time.



Precondition. The signed-in learner has selected a listing from search results; the form includes a proposed start and end.

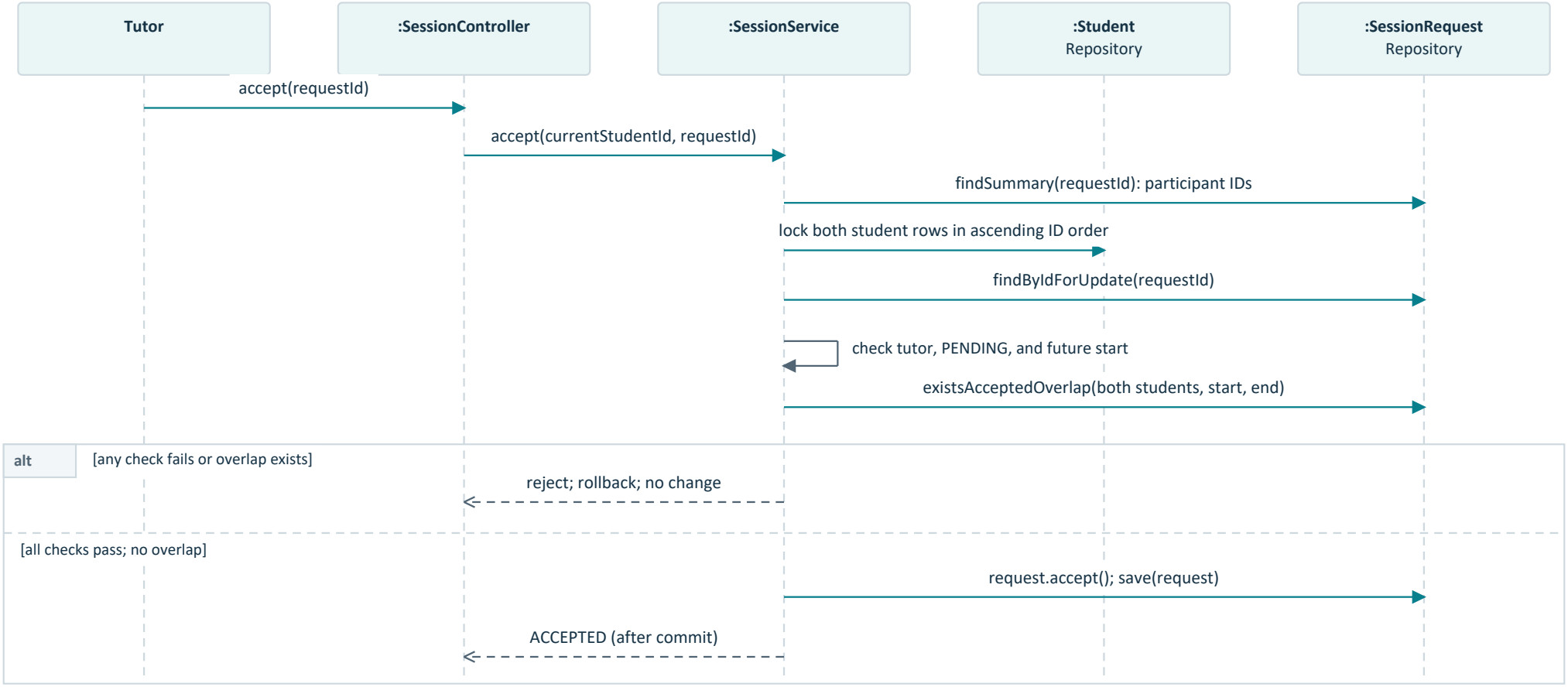
Outcome. The controller redirects to the new pending request. Search itself is a read-only listing query.

Failure handling. BR-02 is checked again. The listing read lock lasts until commit; archive uses a write lock. Invalid input creates no request.

08 / UML SEQUENCE DIAGRAM

Accept without double booking

SD-03 | FR-05 | Unit of Work + participant locks + a fresh conflict check.



Precondition. The tutor submits a request ID. Lookup replies are implicit; a missing request aborts immediately.

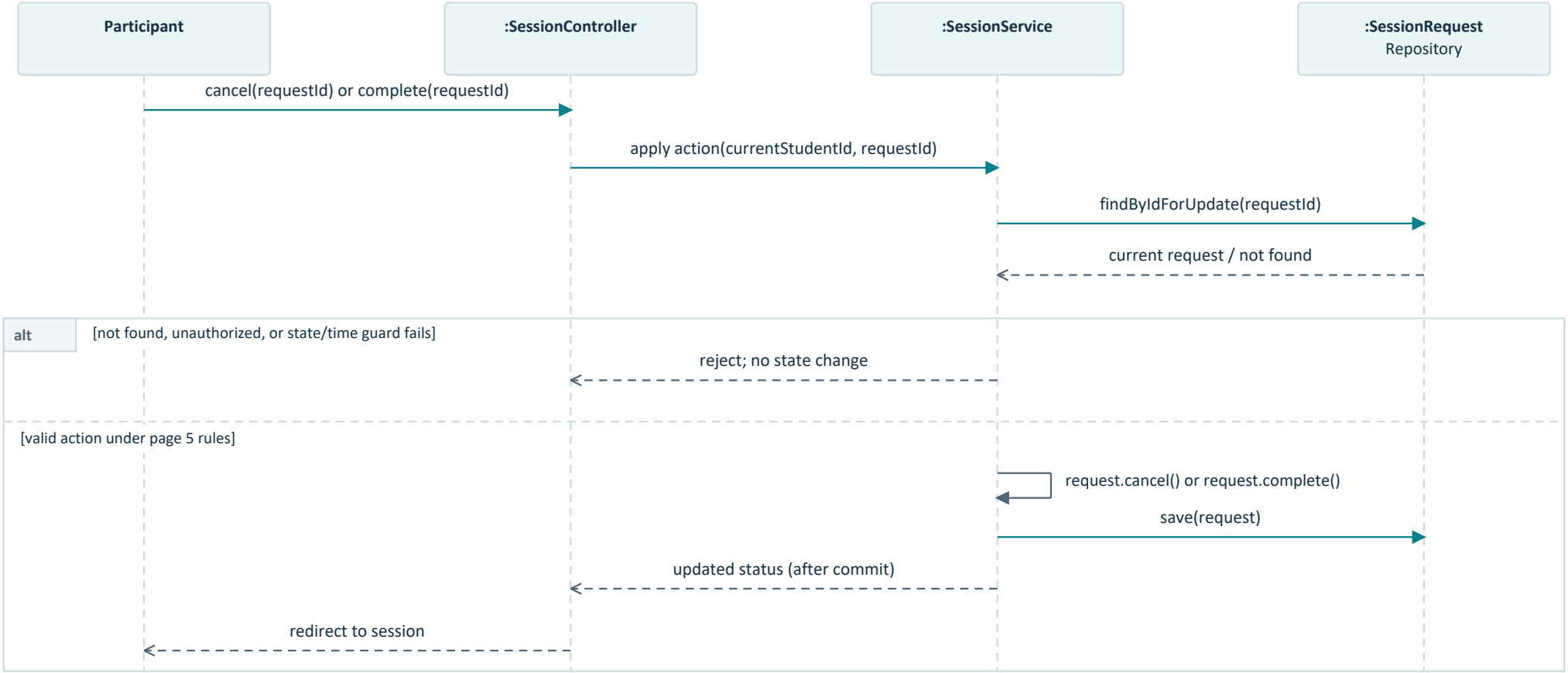
Outcome. The service calls request.accept(), then saves and commits. Both student locks remain held until commit.

Failure handling. All acceptance paths use this lock order and recheck both roles. Invalid actions or conflicts roll back; the controller shows the result.

09 / UML SEQUENCE DIAGRAM

Cancel or complete a session

SD-04 | FR-06 | The service checks the actor; the domain guards the state transition.



Precondition. The signed-in student submits an action from a session page. The server reloads the current state under a lock.

Outcome. An allowed cancellation becomes CANCELLED. A tutor can set COMPLETED at or after the accepted session ends.

Failure handling. A repeated action or a competing update cannot overwrite a terminal state. Decline follows the same transaction structure.

10 / DESIGN RISKS AND PATTERNS

Risks we plan to address

These are anticipated design risks and mitigations, not claims of implemented or tested behavior.

R1 Rules scattered across screens

High impact / medium likelihood

If ownership and status rules are copied into controllers, one path may let a learner accept their own request or let a cancelled session be completed.

Service Layer + domain encapsulation [3]. Route every booking command through SessionService. It checks the authenticated actor and time conditions; SessionRequest methods permit only the transitions on page 5. Controllers cannot set arbitrary statuses.

Tradeoff: an extra application layer. **Planned check:** test every allowed and forbidden actor/state/action combination, including direct HTTP requests and repeat submissions. **Evidence:** CD-01, SD-03, SD-04.

R2 Two bookings accepted at the same time

High impact / medium likelihood

Two requests may both appear conflict-free and then be accepted concurrently, leaving the same learner or tutor booked twice.

Unit of Work + database locking [5, 6, 7]. A service transaction locks both participant rows in ascending ID order, reloads and locks the request, rechecks overlaps, and saves only when free. Use PostgreSQL READ COMMITTED so the post-lock query sees the prior committed booking. Keep the transaction short.

Tradeoff: simultaneous actions can wait. A transaction alone does not prevent this race; all acceptance paths must use the locking protocol. **Planned check:** submit two overlapping acceptances concurrently; exactly one may succeed. **Evidence:** SD-03.

R3 Persistence details leak into business logic

Medium impact / medium likelihood

Duplicated search and booking queries can produce different results between pages, while tests become dependent on a real database for every rule.

Repository pattern [4]. StudentRepository, SkillListingRepository, and SessionRequestRepository define focused lookup, search, lock, and save operations. Services depend on those interfaces; Spring Data JPA supplies the persistence integration.

Tradeoff: interfaces and integration tests must stay aligned. Repositories do not replace database tests. **Planned check:** use service tests for rules and PostgreSQL integration tests for filtering, constraints, transactions, and locks. **Evidence:** SD-01 to SD-04.

11 / VALIDATION AND REFERENCES

How the design will be checked

The checks below define intended acceptance evidence for the implementation phase.

Check	Expected result	Trace
Accounts and ownership	Reject duplicate emails; a student cannot edit another student's profile or listing.	FR-01/02; R1
Search and archive	Search returns only active matching listings; an archived listing rejects new requests while retaining old sessions.	FR-02/03; SD-02
Request validation	Reject self-booking, missing details, past starts, and end times at/before the start.	FR-04; BR-02
Lifecycle and access	Exercise the page 5 transition table; reject wrong actors, terminal-state changes, and premature completion.	FR-05/06; R1
Concurrent acceptance	One of two overlapping accepts succeeds; the other reports a conflict. Check tutor and learner conflicts, including across roles.	FR-05; R2
Boundary and rollback	Back-to-back sessions are allowed; simulated save failure leaves no partial update and releases locks.	BR-03; R2/3

Quality targets

Use labelled, keyboard-accessible forms and readable validation messages. On a local test dataset of 1,000 listings, target a search response within 2 seconds; record the test environment and measurements. Use database foreign keys, a unique email constraint, and an end-after-start constraint. These are proposed targets, not measured results.

Design references

[1] Object Management Group. **UML 2.5.1 specification.** Official source.

[2] Spring. **Serving Web Content with Spring MVC.** Official source.

[3] Martin Fowler. **Service Layer.** Official source.

[4] Martin Fowler catalog. **Repository.** Official source.

[5] Martin Fowler. **Unit of Work.** Official source.

[6] Spring Data JPA. **Transactionality; Locking.** Official source. Locking reference.

[7] PostgreSQL. **Explicit Locking.** Official source. Isolation reference.

Sources accessed 13 October 2026. The design is proposed specifically for SkillSwap; referenced patterns and framework mechanisms inform the choices above.